

Sprint 4 Review Date: Thursday April 30th

Scrum Master for Sprint 4: Aarush Kapoor

Section 0 – Sprint 3 Reality Check (Required)

You must use evidence from your Sprint 3 demo, GitHub activity, or board behavior. Vague answers will not receive full credit.

- What specifically broke during your Sprint 3 demo?
The document upload feature was never created as a task on the GitHub Project Board, meaning it had no owner and no tracked progress throughout the sprint. There are no commits and no board activity related to document upload functionality (which is a continuation of our problem with the board). Because the feature was never broken into tasks, it was effectively nonexistent to us until the demo, at which point it was too late to address. The reason for this is because our sprint tasks were not worded properly. Although the document feature was “included” it was so engrained within the wording that no one would notice unless they went through every possible segment. The specific problem is that when tasks are created, we cant rush any part of the process.
- Which ONE issue is most likely to happen again in Sprint 4?
Board not updated
- What evidence shows this is still a real risk?
(Must reference demo behavior, GitHub activity, or board patterns)
This is still a real risk because this exact failure occurred in both Sprints 2 (personal gallery not developed) and 3 (document upload functionality not created). In both situations, the GitHub board showed active commits on our other features but the missing features had ZERO board activity (no commits made at all). This pattern demonstrates a systemic issue in our sprint planning process and board maintaining process (this is not an isolated incident). Without making formal checks at the start of every development day, this same issue could occur during Sprint 4.

Section 1 – Retrospective Rule Application

- Copy your rule for Sprint 4 from the Sprint 3 Change Plan (exact wording):
At the start of every development day, the Scrum Master (Aarush) will verify that every component of the Sprint Goal has at least one corresponding task on the GitHub Sprint 4 Board with an assigned owner before the team begins ANY development work.
- When does this rule happen during a normal class period?
(Be specific: start, during work time, end of class, etc.)
At the start of every class period (before any development work begins), the Scrum Master (Aarush) checks to make sure every sprint goal component has a corresponding task card with an owner.
- What would I SEE happening if this rule is followed correctly?
You would see the Scrum Master (Aarush) opening the GitHub Project Board on screen

Project 3 – Sprint 4 Planning Document

and reading each sprint goal component out aloud. For each component, a team member would point to a corresponding task card on the Sprint 4 Project Board. If any item has no card, then the team stops and creates one (and assigns an owner) before any development work begins.

- Where will this show up in your workflow? (select all that apply)
 - ☒ Task creation
 - ☐ Task completion
 - ☐ Testing
 - ☒ Board updates
 - ☐ Pull requests
 - ☒ Daily Scrum
- What is ONE way this rule could still fail in Sprint 4?

One way this rule could still fail in Sprint 4 is if a component of the Sprint Goal is worded so vaguely that a sub-feature inside it is missed (this happened with the document upload to personal galleries during Sprint 3). The coverage check would be unable to track such a sub-feature and the sub-feature would never be worked on.
- What happens immediately if this rule is NOT followed?

If this rule is not followed, then all development work must stop. The team cannot write any code until the rule has been followed and the coverage check has been completed. This rule is non-negotiable because of our previous incidents.

Section 2 – Sprint Goal (Expansion + Stability)

Your goal must add new value WITHOUT breaking your existing system.

- Sprint Goal (By the end of Sprint 4, a user will be able to...):

By the end of Sprint 4, a user will be able to upload both images AND documents to their personal galleries; calculate the CO2 savings of choosing to bike instead of driving, using public transit, or walking for a given distance; and log a bike ride to a personal activity log that persists.
- What existing system MUST remain stable?

The Supabase backend (which handles user authentication and data storage for the calendar, Leaflet map, announcements page) must continue to remain stable.
- What would failure look like at demo?

Our demo would fail if: the document upload feature does not work, the CO2 calculator produces incorrect values/is unable to produce values, or if the activity log does not display logged rides across different sessions.

Section 3 – Scope Control (Hard Constraint)

- What IS included in this Sprint (2 – 4 items MAX):

Backlog Item	Why it supports the goal
--------------	--------------------------

Project 3 – Sprint 4 Planning Document

Gamify App	This supports the Sprint 3 Goal component that was unmet (document upload). By allowing users to upload documents/pictures to a personal gallery, it gives users a reason to visit our app and engage with it.
Track CO2	By showing users the environmental impact of choosing to bike over other methods of transportation, we help users understand the benefits of biking and compel them to bike more often (part of Momentum's Product Goal).
Click or tap here to enter text.	Click or tap here to enter text.
Click or tap here to enter text.	Click or tap here to enter text.

- What is NOT included in this Sprint (REQUIRED)
Browsing through available trails (and filtering trails with preferences), sharing rides with the community, personalized recommendations for routes, and displaying money savings associated with biking are all NOT included in this Sprint.
- What are you intentionally delaying and why?
We are intentionally delaying browsing through available trails because they would require the creation of new backend tables, UI pages, and upload of new data. Adding these tasks this sprint would require the team to focus on too many different systems simultaneously, which would risk another sprint where we do not meet our sprint goal.
- Why is this amount of work realistic for your team this Sprint?
This amount of work is realistic for our team this Sprint because the document upload fix is a targeted, small change to an existing system; the CO2 calculator is a feature that does not require the creation of new database systems and entire UI components; the activity log builds on our existing Supabase system and only requires one table and two components (all of which are single class period tasks).

Section 4 – Scope Validation

Do NOT list new backlog items. Use only the items from section 3.

- Which item is the highest risk to your Sprint Goal? Why?
The Activity Calendar is the highest risk. It requires creating a new Supabase table, a ride logging form, and a calendar grid UI, three distinct components that depend on each other sequentially. If the Supabase table schema is not set up correctly in the first task, all subsequent calendar tasks are blocked. Additionally, the calendar grid layout in CSS Grid is a new pattern for our team and could take longer than one period if unexpected rendering issues arise.
- Which item improves stability or user experience?
The Document Upload Fix directly improves stability and user experience. It resolves the

Project 3 – Sprint 4 Planning Document

only incomplete feature from Sprint 3, making the gallery fully functional as originally intended. This also prevents the Sprint 3 demo gap from becoming a recurring issue that compounds into Sprint 5.

- If you had to remove ONE item by Dev Day 3, which would it be? Why?
The Activity Calendar would be removed first. The CO2 calculator and document upload fix are both more directly tied to Momentum's core value proposition (environmental impact tracking and a complete gallery). The calendar, while valuable for engagement, is the newest and most complex feature, removing it preserves the two features most visible to a first-time user at demo.
- Why is this total scope realistic for your team?
Each of the three backlog items has been decomposed into tasks of one class period or less, with clear owners and testable completion criteria. The document fix and CO2 calculator are both self-contained, they do not create dependencies between team members that could cause blocking. The calendar has sequential dependencies but they are internal to Sriram and Yohaana's tasks and have been ordered correctly in the task table. With 11 tasks across 3 features and 4 team members, the load is distributed evenly.

Section 5 – Task System (Strict Execution)

Tasks MUST be ≤ 1 class period.

If the definition of done is vague, the task is invalid.

Add additional rows to this table as necessary by copy and pasting the contents of an existing row and then modifying.

Task	Owner	Backlog Item	Done = (clear criteria)	Includes Testing?	How is it tested?
Fix document upload: update Supabase Storage bucket policy to accept PDF/DOCX MIME types in addition to image/* by modifying the bucket's allowed content types in the Supabase dashboard.	Yohaana	Gamify App	PDF file can be selected and uploaded via the existing pipeline without a console error.	Yes	Upload a .pdf file; confirm a row appears in the Supabase images table with the correct file URL.
Update ImageUpload.tsx: change <code><input accept='image/*'></code> to <code>accept='image/*,application/pdf,.p</code>	Sathkrit h	Gamify App	Done = selecting a PDF shows a	Yes	Test with a .pdf and a .jpg —

Project 3 – Sprint 4 Planning Document

df,.doc,.docx' and update the preview logic to show a file icon for non-image types instead of a broken tag.			file icon preview; selecting an image still shows an image preview.		verify correct preview renders for each in the browser.
End-to-end test: log in, upload one image and one PDF document, confirm both appear in the Gallery grid (image renders as thumbnail, PDF renders as file icon with filename), and persist after a full page refresh.	Sriram	Gamify App	Done = both file types visible in gallery after refresh with no cross-user data leakage	Yes	Log in as two test users, upload different files as each, confirm each user only sees their own files.
Build Calculator.tsx: create a form with inputs for distance (miles) and mode of transport (bike, car, transit, walking) using a dropdown. On submit, compute CO2 emissions using the researched formula and display the result as 'You saved X lbs of CO2 by biking instead of [mode].'	Aarush	Track CO2 + Time and Money saving	Done = entering a distance and selecting a mode displays a CO2 savings figure with no console errors	Yes	Input 10 miles, select Car — verify the output matches the expected formula result calculated manually.
Add a visual comparison bar to Calculator.tsx: render a simple side-by-side bar using Tailwind CSS showing CO2 output for bike vs. the selected mode. Each bar labeled with mode name and CO2 value.	Yohaana	Track CO2 + Time and Money saving	Done = two bars render side-by-side with correct labels and proportional heights in the browser.	Yes	Test with multiple distances and modes to confirm bars resize proportionally and labels update correctly.
Add Calculator to React Router at '/calculator'. Add a navigation link in the app's existing nav bar. Style the page with Tailwind CSS to	Sathkrit h	Track CO2 + Time and	Done = /calculator loads through the	Yes	Navigate to /calculator via nav bar on desktop

Project 3 – Sprint 4 Planning Document

match the app's existing color scheme.		Money saving	nav link with no layout overflow and matches existing app styling.		viewport — confirm no overflow or styling mismatch.
Create a 'rides' table in Supabase: columns id (uuid), user_id (uuid, FK to auth.uid()), date (date), distance_miles (float8), route_name (text), co2_saved_lbs (float8), created_at (timestampz). Insert 2 seed rows manually for testing.	Sriram	Gamify App	Done = 2 seed rows visible in Supabase dashboard under the rides table.	Yes	Verified visually in Supabase table editor.
Build LogRide.tsx: a form with inputs for date, distance (miles), route name, and mode (used to auto-calculate co2_saved). On submit, call supabase.from('rides').insert({..}) with the logged-in user's auth.uid().	Sriram	Gamify App	Done = submitting the form inserts a row into the Supabase rides table visible in the dashboard.	Yes	Submit form with test data; confirm new row appears in Supabase rides table with correct user_id.
Build RideCalendar.tsx: fetch all rides for the logged-in user using supabase.from('rides').select('*').eq('user_id', user.id). Render a monthly calendar grid using CSS Grid where days with logged rides are highlighted with a dot or badge.	Yohaam	Gamify App	Done = calendar renders current month, days with rides are highlighted, and clicking a day shows the ride name and distance.	Yes	Log in with test account that has seed rides; confirm correct days are highlighted and click a highlighted day to verify popup data.
Add RideCalendar and LogRide to React Router at '/calendar'. Add navigation link in nav bar. Add a useEffect refresh trigger so new rides appear on the calendar immediately after logging without a full page reload.	Aarush	Gamify App	Done = /calendar loads through nav link, logging a ride immediately	Yes	Log a ride for today's date and confirm today's cell highlights instantly on

			highlights the correct day on the calendar without refreshing.		the calendar.
--	--	--	--	--	---------------

Section 6 – Testing Integration

- When during the task is testing performed?
Testing is performed immediately upon task completion, before the task is moved to Done on the GitHub board. Each task's Definition of Done includes a specific test condition that must be verified in the browser or Supabase dashboard before the card advances. Testing is not deferred to the end of the sprint.
- Who verifies the task meets Definition of Done BEFORE it moves to Done?
Sriram is the primary tester. For each task, the developer who completed the work demonstrates the Definition of Done criteria to Sriram, who verifies it independently before the card is moved. For tasks assigned to Sriram himself, Aarush (Scrum Master) performs the verification.
- What happens immediately if a test fails?
The task stays in 'In Progress'. The developer who owns the task must diagnose the failure and either fix it within the same class period or split the remaining work into a new task before the next Daily Scrum. The team is notified at the next Daily Scrum so the Scrum Master can offer support or reassign if needed.

Section 7 – Workflow Discipline

- When EXACTLY is the board updated each class?
(Example: last 3 minutes of class before dismissal)
The board is updated twice per class period: once at the start of class (within the first 5-10 minutes) during Aarush's sprint goal coverage check (to confirm all tasks exist and are assigned), and once in the final 5 minutes of class when Sathkrith reviews task progress and moves cards that have met their Definition of Done to the Done column.
- Who checks that the board reflects reality?
Aarush (the Scrum Master) checks the board at the start of each class. Sathkrith performs the end-of-class board review. Both are responsible for flagging discrepancies — if a task appears Done on the board but hasn't met its Definition of Done, it is moved back to the In Progress category immediately.
- How do tasks move (To Do -> In Progress -> Done)?
A task moves from To Do to In Progress the moment the assigned developer makes their first commit toward it. A task moves from In Progress to Done only after the developer has demonstrated the specific Definition of Done criteria to a tester (who cannot be the same person as the developer) and it has been verified.

Project 3 – Sprint 4 Planning Document

- Rule to prevent last-minute work completion:
Before the Daily Scrum each day, Aarush will have a 1–2 minute personal check-in with each developer to ask how their current task is progressing and whether they believe that it is manageable within a single period. If a developer says that they are stuck or unlikely to finish, the team addresses it during the Daily Scrum rather than waiting. Any task that has been In Progress for more than one full class period without moving to Done must be split into smaller sub-tasks at the next Daily Scrum before work continues.

Section 8 – Failure Prevention Plan

- Early warning sign that your Sprint is failing:
A task remains In Progress for more than one class period without a corresponding commit on GitHub, OR a sprint goal component has no task card on the board at the start of a development day.
- Trigger condition (REQUIRED FORMAT):
If [X] is not complete by [time], we will [action]
If the Activity Log's Supabase table and LogRide.tsx tasks are not both marked Done by the end of Dev Day 3, Aarush will flag it at the next Daily Scrum and the team will remove the RideCalendar.tsx display task from sprint scope, focusing only on the ride logging form as the deliverable.
- Who is responsible for making this adjustment?
Aarush (the Scrum Master) is responsible for calling the scope adjustment and communicating it to the team. This decision requires agreement from all team members but Aarush has final authority to trigger it.
- What will you REMOVE from scope if needed?
The calendar grid display (RideCalendar.tsx) will be removed first because the ride logging form (LogRide.tsx) delivers more user value on its own. If further scope cuts are needed, the CO2 visual comparison bar will be removed, leaving only the text-based CO2 savings output.

Section 9 – Expansion Justification

- Why is this the right feature to build now?
The CO2 calculator and the activity log are the right features to build now because they are central to Momentum's core product promise (which is helping users understand and track the environment benefits of choosing to bike). The calculator and the log help create a cohesive MVP that users would actually care about and return to.
- How does it improve user experience or product value?
The CO2 calculator helps users quantify the positive environmental choice that they made; this is the core motivational hook of Momentum for its target audience who care about climate impact. The activity log also helps create a habit-forming loop where users log rides, see their history, and can feel real progress.

Project 3 – Sprint 4 Planning Document

- Why is your system ready to handle this expansion?
(Must reference a specific system improvement from your Sprint 3 retrospective)
Our Supabase authentication infrastructure (built in Sprint 2) and the gallery's per-user data model (built in Sprint 3) are both stable and tested, meaning the new features can build on proven infrastructure rather than introducing risk at the data layer. We also demonstrated in Sprint 3 that task decomposition at the one-period level works: the map feature was completed fully and on time.

Section 10 – Final Commitment Checklist

- ☒ Our rule is observable and enforceable
- ☒ All tasks meet the ≤ 1 class period requirement.
- ☒ Every task includes testing
- ☒ Scope is realistic and constrained
- ☒ Board update expectations are clear
- ☒ We have a plan to detect failure early